

TD7 /TP 7 : Les Fonctions

Exercice 1 :

Écrire une fonction permettant de déterminer si un nombre (entier) passé en paramètre est impair ou non. Cette fonction doit retourner 1 si le nombre est impair et 0 s'il est pair.

Tester la fonction dans un programme interactif. C'est le programme principal qui affichera si le nombre est pair ou impair.

```
Code : #####  
#include <stdio.h>  
int est_pair(int nombre) {  
    return nombre % 2 != 0; // Retourne 1 si impair, 0 si pair  
}  
int main() {  
    int nombre;  
    printf("Entrez un nombre entier : ");  
    scanf("%d", &nombre);  
    if (est_pair(nombre)) {  
        printf("Le nombre %d est impair.\n", nombre);  
    } else {  
        printf("Le nombre %d est pair.\n", nombre);  
    }  
    return 0;  
}  
#####
```

Exercice 2 :

- Écrire une fonction qui affichera le menu suivant :

```
*****MENU*****  
1---->Racine carrée  
2---->carrée  
3---->cube  
4---->Inverse  
5---->Somme  
6---->Différence  
7---->Produit  
8--->Division entiere
```

9---->Quitter

Entrez votre choix (1-9) :

- Ecrire une fonction qui calculera la Racine carrée.
- Ecrire une fonction qui calculera le carrée.
- Ecrire une fonction qui calculera le cube.
- Ecrire une fonction qui calculera l'inverse.
- Ecrire une fonction qui calculera la somme.
- Ecrire une fonction qui calculera la différence.
- Ecrire une fonction qui calculera le produit.
- Ecrire une fonction qui calculera la division entière.
- Ecrire un programme utilisant les fonctions précédentes.

Code :

```
#####  
#include <stdio.h>  
#include <math.h>  
void afficher_menu() {  
    printf("*****MENU*****\n");  
    printf("1---->Racine carrée\n");  
    printf("2---->Carré\n");  
    printf("3---->Cube\n");  
    printf("4---->Inverse\n");  
    printf("5---->Somme\n");  
    printf("6---->Différence\n");  
    printf("7---->Produit\n");  
    printf("8---->Division entière\n");  
    printf("9---->Quitter\n");  
    printf("*****\n");  
    printf("Entrez votre choix (1-9) : ");  
}  
double racine_carree(double nombre) {  
    return sqrt(nombre);  
}  
double carre(double nombre) {  
    return nombre * nombre;  
}  
double cube(double nombre) {  
    return nombre * nombre * nombre;  
}
```

```

double inverse(double nombre) {
    if (nombre != 0) {
        return 1 / nombre;
    } else {
        printf("Erreur : Division par zéro !\n");
        return 0;
    }
}

double somme(double a, double b) {
    return a + b;
}

double difference(double a, double b) {
    return a - b;
}

double produit(double a, double b) {
    return a * b;
}

int division_entiere(int a, int b) {
    if (b != 0) {
        return a / b;
    } else {
        printf("Erreur : Division par zéro !\n");
        return 0;
    }
}

int main() {
    int choix;
    double nombre, nombre2;

    do {
        afficher_menu();
        scanf("%d", &choix);
        switch (choix) {
            case 1:
                printf("Entrez un nombre : ");
                scanf("%lf", &nombre);
                printf("La racine carrée de %.2lf est %.2lf\n", nombre,
racine_carree(nombre));
                break;
            case 2:
                printf("Entrez un nombre : ");
                scanf("%lf", &nombre);

```

```

        printf("Le carré de %.2lf est %.2lf\n", nombre, carre(nombre));
        break;
case 3:
    printf("Entrez un nombre : ");
    scanf("%lf", &nombre);
    printf("Le cube de %.2lf est %.2lf\n", nombre, cube(nombre));
    break;
case 4:
    printf("Entrez un nombre : ");
    scanf("%lf", &nombre);
    if (nombre != 0) {
        printf("L'inverse de %.2lf est %.2lf\n", nombre, inverse(nombre));
    }
    break;
case 5:
    printf("Entrez deux nombres : ");
    scanf("%lf %lf", &nombre, &nombre2);
    printf("La somme de %.2lf et %.2lf est %.2lf\n", nombre, nombre2,
somme(nombre, nombre2));
    break;
case 6:
    printf("Entrez deux nombres : ");
    scanf("%lf %lf", &nombre, &nombre2);
    printf("La différence entre %.2lf et %.2lf est %.2lf\n", nombre,
nombre2, difference(nombre, nombre2));
    break;
case 7:
    printf("Entrez deux nombres : ");
    scanf("%lf %lf", &nombre, &nombre2);
    printf("Le produit de %.2lf et %.2lf est %.2lf\n", nombre, nombre2,
produit(nombre, nombre2));
    break;
    case 8:
        printf("Entrez deux nombres entiers : ");
        int entier1, entier2;
        scanf("%d %d", &entier1, &entier2);
        if (entier2 != 0) {
            printf("La division entière de %d par %d est %d\n", entier1,
entier2, division_entiere(entier1, entier2));
        }
        break;
case 9:

```

```

        printf("Au revoir !\n");
        break;
    default:
        printf("Choix invalide. Veuillez entrer un numéro entre 1 et 9.\n");
    }
} while (choix != 9);
return 0;
}
#####

```

Exercice 3 :

Ecrire la fonction NCHIFFRES du type int qui obtient une valeur entière N (positive ou négative) du type long comme paramètre et qui fournit le nombre de chiffres de N comme résultat.

Ecrire un petit programme qui teste la fonction NCHIFFRES.

Exemple :

Introduire un nombre entier : 6457392

Le nombre 6457392 à 7 chiffres.

Code :

```

#include <stdio.h>
int NCHIFFRES(long N) {
    int count = 0;
    // Si N est zéro, il a exactement un chiffre
    if (N == 0) {
        return 1;
    }
    // Ignorer le signe négatif
    if (N < 0) {
        N = -N;
    }
    while (N != 0) {
        count++;
        N /= 10; // Retirer le dernier chiffre
    }

    return count;
}

int main() {
    long nombre;
    printf("Entrez un nombre entier : ");
    scanf("%ld", &nombre);
}

```

```

        printf("Le nombre de chiffres de %ld est : %d\n", nombre,
NCHIFFRES(nombre));
        return 0;
    }

```

#####

Exercice 4 :

En mathématiques, on définit la fonction factorielle de la manière suivante:

$0! = 1$

$n! = n*(n-1)*(n-2)* \dots * 1$ (pour $n > 0$)

Ecrire une fonction FACT du type **double** qui reçoit la valeur N (type **int**) comme paramètre et qui fournit la factorielle de N comme résultat. Ecrire un petit programme qui teste la fonction FACT.

#####

Code :

Méthode 1 :

```

#include <stdio.h>
double FACT(int N) {
    if (N < 0) {
        printf("Erreur : La factorielle n'est pas définie pour les nombres négatifs.\n");
        return -1;
    }
    double resultat = 1;
    for (int i = 1; i <= N; i++) {
        resultat *= i;
    }
    return resultat;
}

```

Méthode 2 :

```

#include <stdio.h>

```

```

double FACT(int N) {
    if (N < 0) {
        printf("Erreur : La factorielle n'est pas définie pour les nombres négatifs.\n");
        return -1; // Valeur d'erreur
    }

```

```

        if (N == 0 || N == 1) {
            return 1;
        }

```

```

return N * FACT(N - 1);
}

```

```

int main() {
    int nombre;
    printf("Entrez un nombre entier : ");
    scanf("%d", &nombre);
    double factorielle = FACT(nombre);
    if (factorielle != -1) {
        printf("La factorielle de %d est : %.0lf\n", nombre, factorielle);
    }
    return 0;
}

```

#####

Exercice 5 :

Pour faire cet exercice, utiliser les fonctions prédéfinies suivantes :

int rand(void): une fonction standard de la bibliothèque <stdlib.h> retourne un entier pseudo-aléatoire dans l'intervalle [0, RAND_MAX].

RAND_MAX : une constante dont la valeur par défaut peut varier selon les implémentations, mais sa valeur sera moins égale à 32767.

1. Créer une fonction **De**, qui prend en entrée 1 entier qui correspond au nombre de faces d'un dé et qui retourne aléatoirement un numéro de face.

Exemple **de(6)** correspond à un lancé de dé de 6, donc un retour entre 1 et 6.

2. Créer une fonction **Des**, qui prend en entrée 2 entiers - Nombre de dés à lancer. - Nombre de faces par dé.

Cette fonction retourne alors la somme des résultats de chaque dé.

Code : #####

```

1)
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

```

// Fonction qui génère un numéro de face aléatoire pour un dé avec un nombre de faces spécifié.

```

int De(int faces) {
    return rand() % faces + 1; // Génère un nombre entre 1 et 'faces'
}

```

```

int main() {
    // Initialiser le générateur de nombres aléatoires avec l'heure actuelle
    srand(time(NULL));

    int faces = 6;
    printf("Le résultat du dé à %d faces est: %d\n", faces, De(faces));
    return 0;
}

2)
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
// Fonction qui génère un numéro de face aléatoire pour un dé avec un nombre de faces
spécifié.
int De(int faces) {
    return rand() % faces + 1;
}
// Fonction qui lance plusieurs dés avec un nombre de faces donné
void Des(int nombre_de_des, int faces_par_de) {
    printf("Résultats des %d dés à %d faces :\n", nombre_de_des, faces_par_de);
    for (int i = 0; i < nombre_de_des; i++) {
        printf("Dé %d: %d\n", i + 1, De(faces_par_de)); // Appel à la fonction De pour
chaque dé
    }
}

int main() {

    srand(time(NULL));

    int nombre_de_des = 3;
    int faces_par_de = 6;
    Des(nombre_de_des, faces_par_de);
    return 0;
}

```